Comparare — Exerciții

Ștefan · `IComparable<T>` și `IComparer<T>`

Unde scrii: fișierele noi sunt deja în `Exercitii/` : `Sectiune7Comparare.cs` (metodele cu numele din tabel, datele de pornire sunt declarate), `Elev.cs` și `Bicicleta.cs` . În `Main` adaugi tu, după `Sectiune6();` , linia `Sectiune7();` . Noțiunile: `IComparable<T>` , `CompareTo` , `IComparer<T>` , `Compare` — în **COLECTII.pdf** (pagina nouă).
Ieșirea: colecția o afișezi cu `string.Join(", ", c)` , un `bool` sau un număr cum sunt; excepția o prinzi cu `try/catch (Exception e)` și afișezi `e.GetType().Name` . Celula goală = o notezi tu după ce rulezi.

7. IComparable<T> și IComparer<T>

Cele două liste din tabel au aceeași clasă `Elev` (`Nume` + `Nota` , afișat `Ana(9)`), aflată în `Exercitii/Elev.cs` ; la 7.1 o modifici chiar pe ea. Clasele noi (`ComparerDupaNume` etc.) le scrii fiecare într-un fișier nou din `Exercitii/` , cu numele clasei. Noțiunile: `IComparable<T>` , `CompareTo` , `IComparer<T>` , `Compare` — în **COLECTII.pdf**.

```
List<Elev> lista = new List<Elev> { new Elev("Ana", 9), new Elev("Ion", 6), new Elev("Maria", 10), new Elev("Dan", 7) };
```

#	Cerința	Ieșire
7.1	**`FaElevulComparabil`** Fă `Elev` să implementeze `IComparable<Elev>` (`CompareTo`): ordinea lui naturală e după `Nota` , crescător. Apoi `lista.Sort()` ; afișează lista.	`Ion(6), Dan(7), Ana(9), Maria(10)`
7.2	**`SortedSetAreNevoieDeComparabil`** După ce ai făcut 7.1 (`Elev` comparabil): pune `elevi` (aceiași patru) într-un `SortedSet<Elev>` și afișează-l. Apoi, într-un `try/catch` , creează un `SortedSet<Bicicleta>` gol și **adaugă două biciclete** (`pegas` și `cube` , din cod; `Bicicleta` nu e comparabilă); afișează numele excepției și, dacă există, al `InnerException` .	
7.3	**`ComparerDupaNume`** Scrie clasa `ComparerDupaNume : IComparer<Elev>` (`Compare`), după `Nume` alfabetic, fără să atingi `Elev` . Apoi `lista.Sort(new ComparerDupaNume())` ; afișează lista.	`Ana(9), Dan(7), Ion(6), Maria(10)`
7.4	**`ComparerDupaNotaDescrescator`** La fel, clasa `ComparerDupaNotaDescrescator : IComparer<Elev>` : nota cea mai mare prima. `lista.Sort(...)` cu ea; afișează lista.	`Maria(10), Ana(9), Dan(7), Ion(6)`
7.5	**`SortedDictionaryCuComparer`** Scrie `ComparerDupaLungime : IComparer<string>` : întâi după lungime, la lungime egală alfabetic (`string.Compare(a, b, StringComparison.Ordinal)`). Pune cheile din `chei` (valoarea = lungimea) întâi într-un `new SortedDictionary<string, int>(new ComparerDupaLungime())` , apoi într-unul fără comparer; afișează cheile fiecăruia, câte un rând.	`ou, unt, lapte, paine` `lapte, ou, paine, unt`
7.6	**`CompareToEgalInseamnaDuplicat`** `ana` și `vlad` au aceeași notă. Adaugă `ana` , apoi `vlad` într-un `SortedSet<Elev>` (cu `CompareTo` de la 7.1): afișează ce întoarce al doilea `Add` și mulțimea. Apoi pune-i într-un `HashSet<Elev>` și afișează `Count` .	